Practical – 10

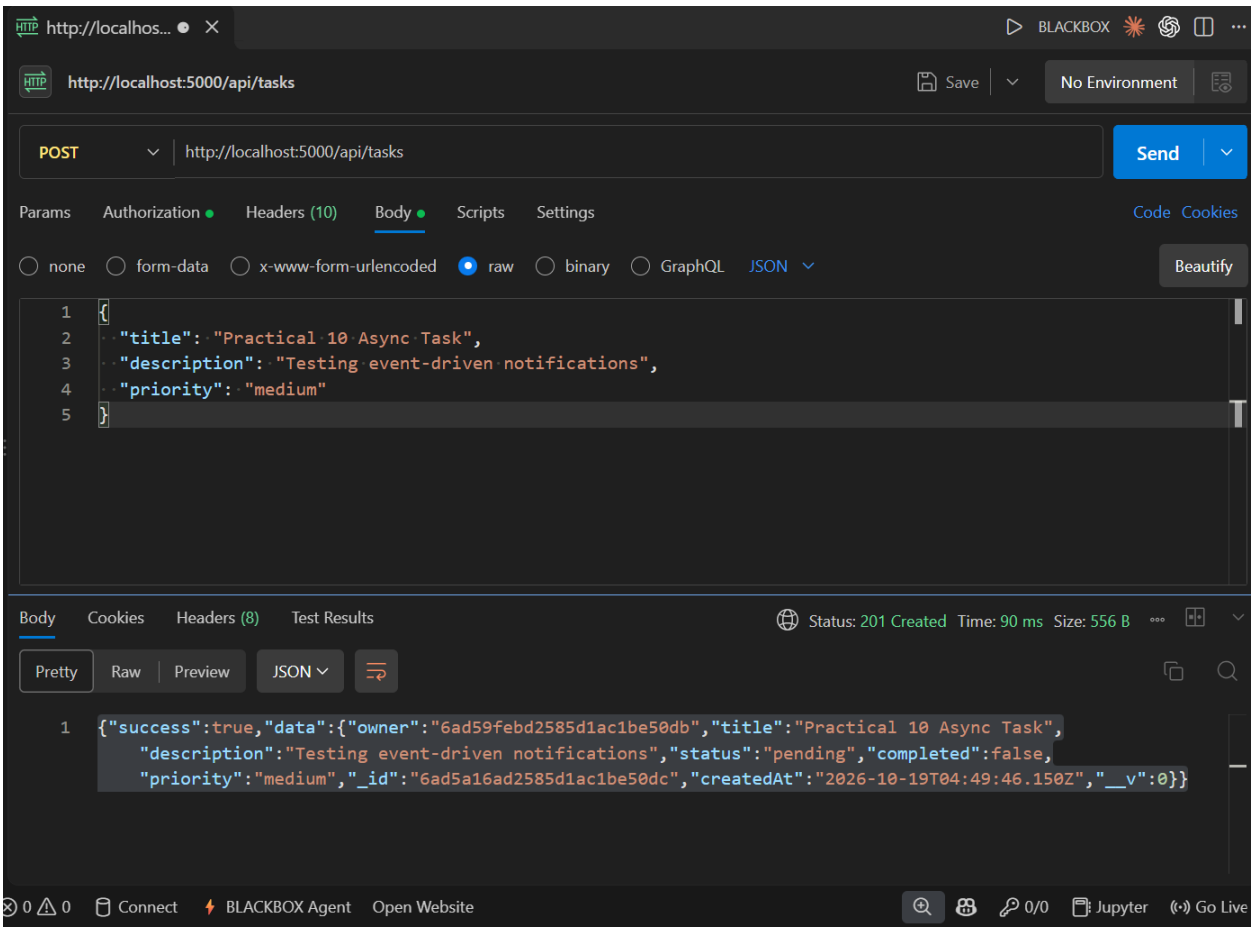
Aim:

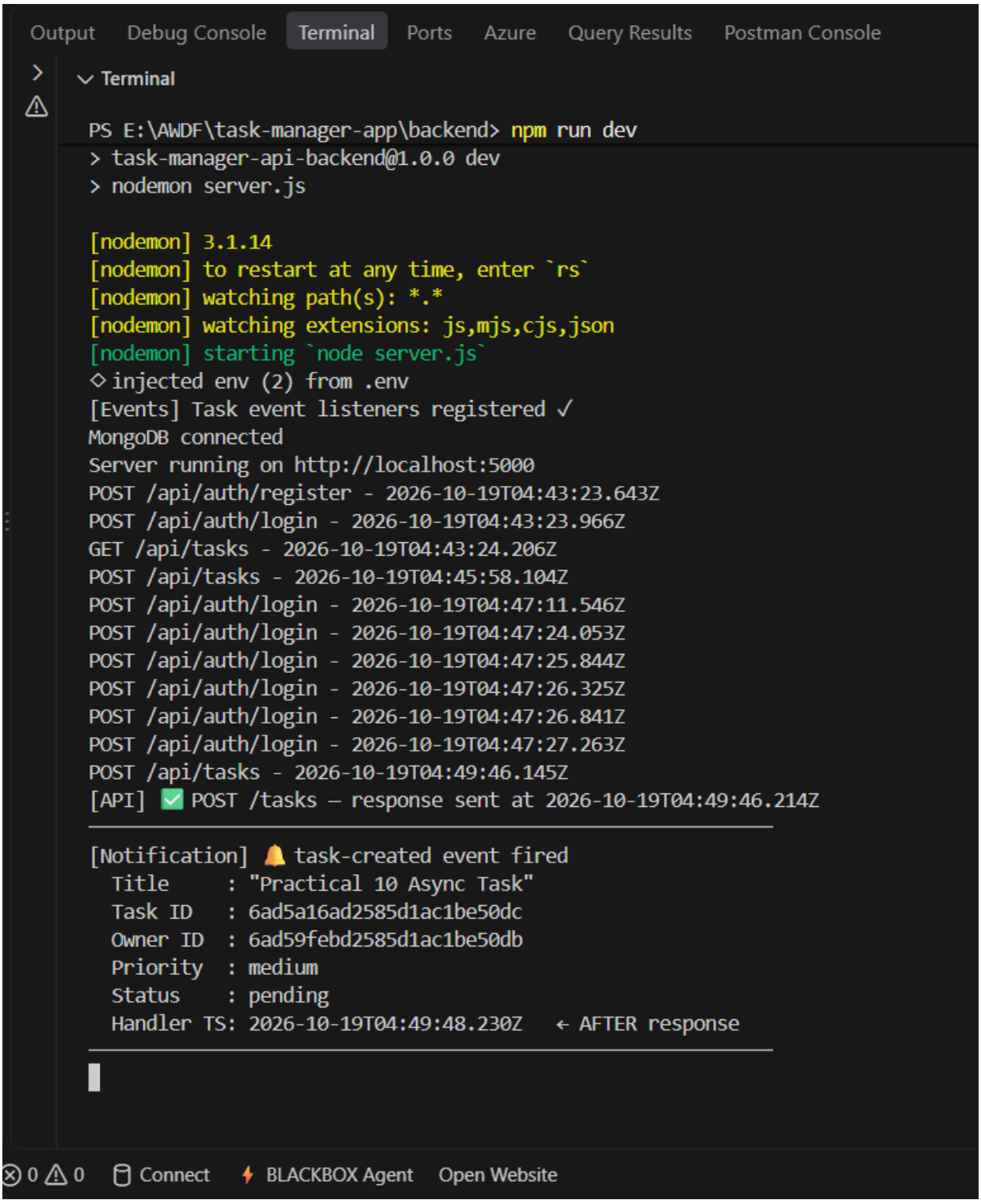
To implement asynchronous background processing using Node.js native EventEmitter without external dependencies.

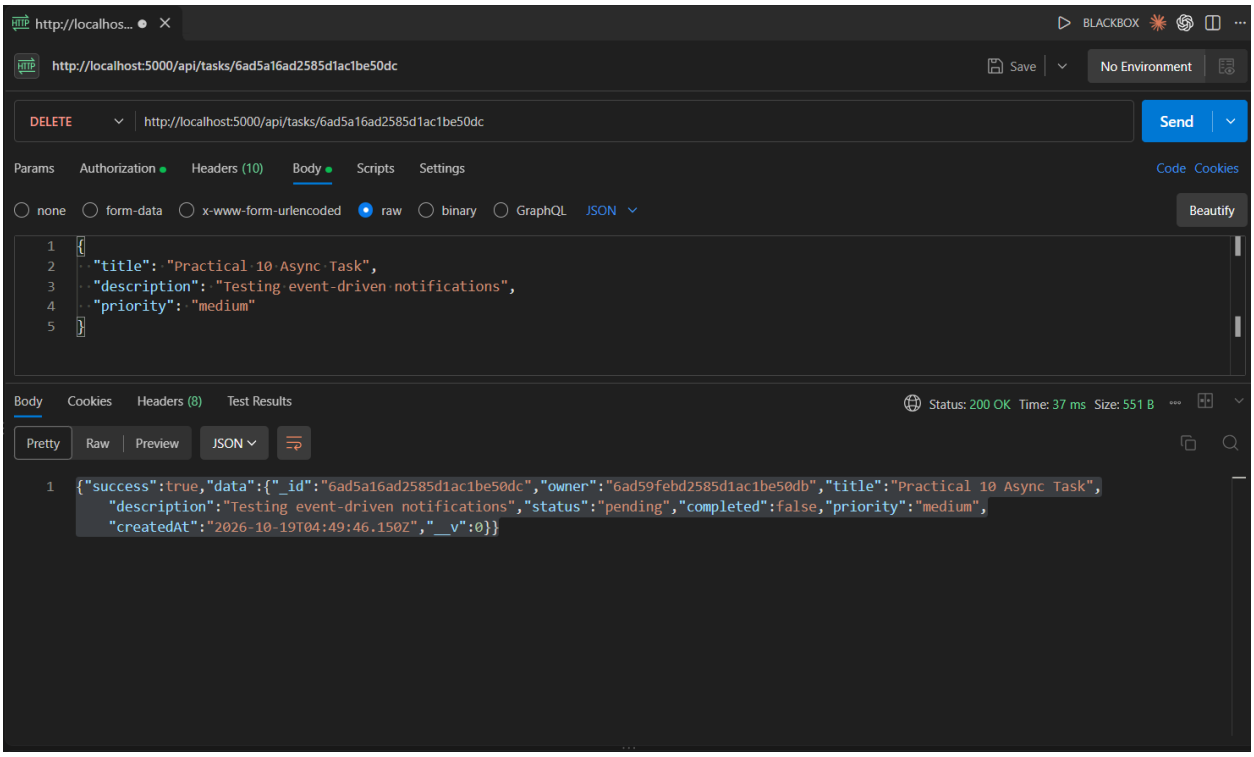
Content:

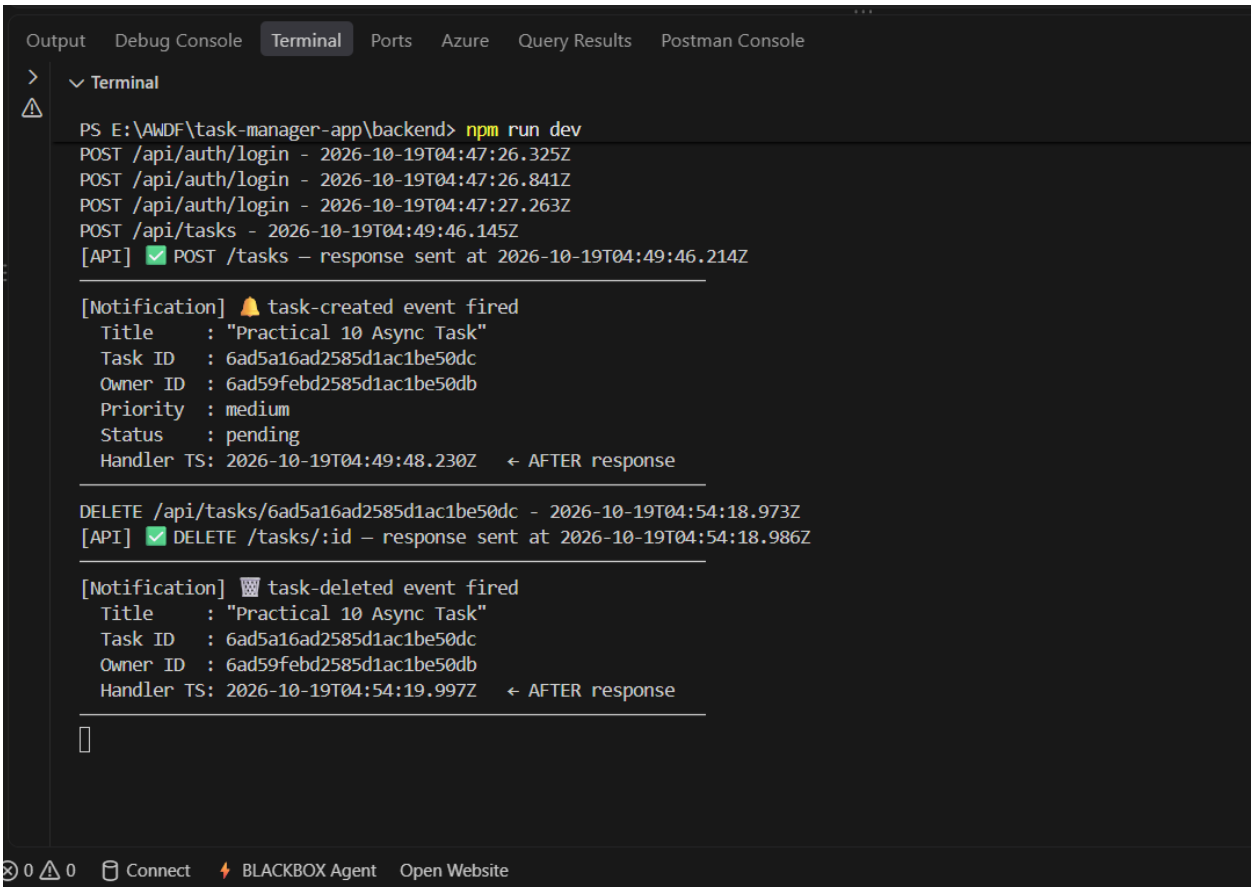
After completing this practical, I will be able to improve the performance of a Task Management application using asynchronous processing with Node.js EventEmitter. I will learn how to create events, handle notifications in the background, and send API responses without waiting for the notification process to finish. I will also learn how to use timestamps to check the difference between API response time and notification time. This practical will help me understand event-driven architecture and build faster and more responsive applications.

Screen Shot:




Output
Debug Console
Terminal
Ports
Azure
Query Results
Postman Console
Terminal
PS E:\AWDF\task-manager-app\backend> npm run dev
> task-manager-api-backend@1.0.0 dev
> nodemon server.js
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
◇ injected env (2) from .env
[Events] Task event listeners registered ✓
MongoDB connected
Server running on http://localhost:5000
POST /api/auth/register - 2026-10-19T04:43:23.643Z
POST /api/auth/login - 2026-10-19T04:43:23.966Z
GET /api/tasks - 2026-10-19T04:43:24.206Z
POST /api/tasks - 2026-10-19T04:45:58.104Z
POST /api/auth/login - 2026-10-19T04:47:11.546Z
POST /api/auth/login - 2026-10-19T04:47:24.053Z
POST /api/auth/login - 2026-10-19T04:47:25.844Z
POST /api/auth/login - 2026-10-19T04:47:26.325Z
POST /api/auth/login - 2026-10-19T04:47:26.841Z
POST /api/auth/login - 2026-10-19T04:47:27.263Z
POST /api/tasks - 2026-10-19T04:49:46.145Z
[API] ✅ POST /tasks — response sent at 2026-10-19T04:49:46.214Z
[Notification] 🔔 task-created event fired
Title : "Practical 10 Async Task"
Task ID : 6ad5a16ad2585d1ac1be50dc
Owner ID : 6ad59febd2585d1ac1be50db
Priority : medium
Status : pending
Handler TS: 2026-10-19T04:49:48.230Z ← AFTER response
0 0
Connect
BLACKBOX Agent
Open Website


http://localhos...
BLACKBOX
http://localhost:5000/api/tasks/6ad5a16ad2585d1ac1be50dc
Save
No Environment
DELETE
http://localhost:5000/api/tasks/6ad5a16ad2585d1ac1be50dc
Send
Params
Authorization
Headers (10)
Body
Scripts
Settings
Code
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"title": "Practical 10 Async Task",
"description": "Testing event-driven notifications",
"priority": "medium"
}
Body
Cookies
Headers (8)
Test Results
Status: 200 OK
Time: 37 ms
Size: 551 B
Pretty
Raw
Preview
JSON
{"success":true,"data":{"_id":"6ad5a16ad2585d1ac1be50dc","owner":"6ad59febd2585d1ac1be50db","title":"Practical 10 Async Task",
"description":"Testing event-driven notifications","status":"pending","completed":false,"priority":"medium",
"createdAt":"2026-10-19T04:49:46.150Z","__v":0}}


Output
Debug Console
Terminal
Ports
Azure
Query Results
Postman Console
Terminal
PS E:\AWDF\task-manager-app\backend> npm run dev
POST /api/auth/login - 2026-10-19T04:47:26.325Z
POST /api/auth/login - 2026-10-19T04:47:26.841Z
POST /api/auth/login - 2026-10-19T04:47:27.263Z
POST /api/tasks - 2026-10-19T04:49:46.145Z
[API] ✅ POST /tasks — response sent at 2026-10-19T04:49:46.214Z
[Notification] 🔔 task-created event fired
Title : "Practical 10 Async Task"
Task ID : 6ad5a16ad2585d1ac1be50dc
Owner ID : 6ad59febd2585d1ac1be50db
Priority : medium
Status : pending
Handler TS: 2026-10-19T04:49:48.230Z ← AFTER response
DELETE /api/tasks/6ad5a16ad2585d1ac1be50dc - 2026-10-19T04:54:18.973Z
[API] ✅ DELETE /tasks/:id — response sent at 2026-10-19T04:54:18.986Z
[Notification] 🗑 task-deleted event fired
Title : "Practical 10 Async Task"
Task ID : 6ad5a16ad2585d1ac1be50dc
Owner ID : 6ad59febd2585d1ac1be50db
Handler TS: 2026-10-19T04:54:19.997Z ← AFTER response
0 0
Connect
BLACKBOX Agent
Open Website

Conclusion:

This practical demonstrates the implementation of asynchronous processing in a Task Management application using Node.js, Express.js, MongoDB, and EventEmitter. It includes creating events when a new task is added, handling notifications in the background, and sending API responses without waiting for the notification process to finish. It also covers adding a task-deleted event, displaying notification details, and comparing timestamps to understand how asynchronous processing works. This practical helps improve application performance and makes the backend faster and more efficient.